

PyChat

Chat Multi-Cliente com Sockets TCP em Python

IFSP São Carlos — Disciplina: Redes de Computadores

Este documento descreve a aplicação **PyChat**, um sistema de chat em tempo real implementado com sockets **TCP** em Python. Desenvolvido como projeto prático da disciplina de Redes de Computadores, o sistema demonstra na prática os conceitos de sockets, modelo cliente-servidor e comunicação em rede.

1. Descrição da Aplicação

O PyChat é um sistema de chat em tempo real que permite a comunicação simultânea entre múltiplos usuários conectados a um servidor central. A aplicação foi desenvolvida utilizando o módulo **socket** nativo do Python, com protocolo **TCP (Transmission Control Protocol)**, que garante entrega confiável e ordenada das mensagens.

Funcionalidades principais:

- **Chat em grupo:** mensagens enviadas por um usuário são recebidas por todos os outros conectados.
- **Mensagens privadas:** o comando `/msg` permite comunicação direta entre dois usuários.
- **Listagem de usuários:** o comando `/users` exibe todos os participantes online.
- **Apelidos únicos:** o servidor rejeita conexões com apelido já em uso.
- **Multi-threading:** o servidor atende múltiplos clientes simultaneamente, sem bloqueio.
- **Notificações:** todos os usuários são avisados quando alguém entra ou sai do chat.

2. Arquitetura do Sistema

A aplicação segue o modelo **Cliente-Servidor**. O servidor roda de forma contínua aguardando conexões. Cada cliente que se conecta recebe uma thread exclusiva no servidor, permitindo comunicação paralela sem bloqueio.

Arquivos do projeto:

Arquivo	Função
<code>servidor.py</code>	Servidor TCP — aceita conexões, gerencia clientes e distribui mensagens

Arquivo	Função
cliente.py	Cliente TCP — conecta ao servidor, envia e recebe mensagens
README.md	Instruções de instalação, execução e link do vídeo

Fluxo de comunicação:

1. O servidor é iniciado e fica escutando na porta 5000.
2. O cliente estabelece uma conexão TCP com o servidor (connect).
3. O servidor envia o sinal NICK solicitando o apelido.
4. O cliente responde com seu apelido escolhido.
5. O servidor verifica unicidade do apelido e confirma a conexão.
6. O servidor cria uma thread exclusiva para o novo cliente.
7. Mensagens são trocadas até que o cliente envie /sair ou se desconecte.

3. Tecnologias Utilizadas

Módulo / Tecnologia	Versão	Uso no projeto
Python	3.8+	Linguagem de programação
socket	nativo	Criação e gerenciamento dos sockets TCP
threading	nativo	Concorrência — uma thread por cliente no servidor
datetime	nativo	Timestamps nos logs do servidor
select	nativo	Verificação não-bloqueante de dados disponíveis
sys	nativo	Controle de saída do programa

Por que TCP? O protocolo TCP foi escolhido porque em um chat é fundamental que todas as mensagens cheguem ao destino na ordem correta. O TCP garante essas propriedades através do seu mecanismo de confirmação (ACK) e controle de fluxo, ao contrário do UDP, que não oferece tais garantias.

4. Manual do Usuário

4.1 Pré-requisitos

- Python 3.8 ou superior instalado na máquina.
- Nenhuma biblioteca externa necessária — apenas a biblioteca padrão do Python.
- Em caso de múltiplas máquinas, todas devem estar na mesma rede local ou com a porta 5000 acessível.

4.2 Iniciando o Servidor

Abra um terminal na pasta do projeto e execute o comando abaixo. O servidor deve permanecer aberto enquanto o chat estiver em uso:

```
python servidor.py
```

O terminal exibirá logs de todas as conexões, desconexões e mensagens trocadas.

4.3 Conectando como Cliente

Em um terminal separado (mesma máquina ou outra máquina da rede), execute:

```
python cliente.py
```

O sistema solicitará um apelido. Digite um nome único e pressione Enter. Se o apelido já estiver em uso, a conexão será recusada.

Conectando em outra máquina: edite a linha `HOST = '127.0.0.1'` no arquivo `cliente.py`, substituindo pelo IP do servidor. Exemplo: `HOST = '192.168.1.10'`.

4.4 Comandos Disponíveis

Comando	Descrição	Exemplo
	Envia mensagem para todos os usuários	Oi pessoal!
/msg	Mensagem privada para um usuário específico	/msg Ana Oi!
/users	Lista todos os usuários conectados	/users
/ajuda	Exibe os comandos disponíveis	/ajuda
/sair	Desconecta e encerra o cliente	/sair

5. Exemplo de Sessão

Terminal do Servidor:

```
[10:00:01] Servidor PyChat iniciado em 0.0.0.0:5000
[10:00:01] Aguardando conexoes...
[10:00:10] Nova conexao de 127.0.0.1:54321
[10:00:10] Bruno entrou no chat.
[10:00:15] Nova conexao de 127.0.0.1:54322
[10:00:15] Ana entrou no chat.
```

[10:00:20] Bruno: Oi pessoal!

[10:00:25] Ana: Ola Bruno!

Terminal do Cliente — Bruno:

Digite seu apelido: Bruno

[Sistema] Conectado ao servidor 127.0.0.1:5000 como 'Bruno'.

[Sistema] Bem-vindo(a), Bruno!

[Sistema] Ana entrou no chat!

Oi pessoal!

[Voce]: Oi pessoal!

[Ana]: Ola Bruno!

/msg Ana Voce viu a tarefa de redes?

[Privado para Ana]: Voce viu a tarefa de redes?

Terminal do Cliente — Ana:

Digite seu apelido: Ana

[Sistema] Conectado ao servidor 127.0.0.1:5000 como 'Ana'.

[Sistema] Bem-vindo(a), Ana!

[Bruno]: Oi pessoal!

Ola Bruno!

[Voce]: Ola Bruno!

[Privado de Bruno]: Voce viu a tarefa de redes?

6. Conceitos de Redes Aplicados

Socket TCP:

Um socket é um ponto de comunicação que permite a troca de dados entre processos, seja na mesma máquina ou em máquinas distintas conectadas em rede. No PyChat, cada cliente possui um socket TCP individual que mantém uma conexão persistente com o servidor até que uma das partes encerre a ligação.

Modelo Cliente-Servidor:

O servidor permanece em execução aguardando passivamente conexões (`listen` / `accept`). Os clientes tomam a iniciativa de conectar (`connect`). Essa assimetria é a característica fundamental do modelo cliente-servidor.

Threading e Concorrência:

Para que o servidor possa atender múltiplos clientes ao mesmo tempo sem que um bloqueie o outro, é criada uma thread separada para cada cliente conectado. As threads compartilham as listas de clientes e apelidos, protegidas por um `threading.Lock` para evitar condições de corrida.

Encapsulamento de dados:

As mensagens trafegam como bytes (`encode/decode UTF-8`). O protocolo da aplicação é simples: o sinal `NICK` no handshake e prefixos textuais como `/msg` para identificar comandos especiais.